LLM Coding Exercise 1

CNN Digit Classifier

Modules: Activations, Layers, Loss, and Training Results

1. Activations (modules/activation.py)

1.1 ReLU

File: `activation.py` — Lines 21–56

Equation:

$$ReLU(x) = max(0, x)$$

Forward pass code:

```
return np.maximum(0, inputs)
```

Backward pass code:

```
return {"d_out": d_outputs * (self.inputs > 0)}
```

Why it is used: ReLU introduces non-linearity without saturating (unlike sigmoid/tanh) and produces sparse activations — all negative inputs become zero. This makes gradients efficient and avoids the vanishing-gradient problem.

Gradient rule: The derivative is 1 where x > 0 and 0 otherwise, so the upstream gradient `d_outputs` is masked element-wise by `(self.inputs > 0)`.

1.2 Sigmoid

File: `activation.py` — Lines 59–96
Note: Implemented but not used in main.py.

Equation:

$$\sigma(x) = 1 / (1 + e^{(-x)})$$

Forward pass code:

```
return 1 / (1 + np.exp(-inputs))
```

Backward — derivative:

$$d\sigma/dx = \sigma(x) \cdot (1 - \sigma(x))$$

Backward pass code:

```
sig = 1 / (1 + np.exp(-self.inputs))
return {"d_out": d_outputs * sig * (1 - sig)}
```

1.3 Softmax

File: `activation.py` — Lines 99–140 — used as the final layer

Equation (for class logits z):

$$softmax(z_i) = e^{\wedge}(z_i) \,/\, \Sigma_j\, e^{\wedge}(z_j)$$

Forward pass code (numerically stable):

```
exp_shifted = np.exp(inputs - np.max(inputs, axis=-1, keepdims=True))
return exp_shifted / np.sum(exp_shifted, axis=-1, keepdims=True)
```

Stability trick: The inputs are shifted by their row-wise maximum (`z_i ← z_i - max(z)`) before exponentiation to prevent numerical overflow.

Why softmax: Converts raw logits into a proper probability distribution over the 10 digit classes (values sum to 1).

Backward pass: Computes the gradient w.r.t. inputs using the Softmax structure, avoiding construction of the full Jacobian:

```
softmax = self.forward(self.inputs)
dot = np.sum(d_outputs * softmax, axis=-1, keepdims=True)
d_inputs = softmax * (d_outputs - dot)
return {"d_out": d_inputs}
```

2. Layers (modules/layers.py)

The model pipeline is: `ConvolutionLayer → ReLU → Flatten → LinearLayer → Softmax`

2.1 ConvolutionLayer

Initialization (Kaiming / He Scale)

File: `layers.py` — Lines 27–45

Weight initialisation formula:

$$std = \sqrt{(2 / fan_in)} \quad where\ fan_in = in_channels \times kernel_size^2$$

Code:

```
fan_in = in_channels * kernel_size * kernel_size
std = np.sqrt(2.0 / fan_in)
self.weights = np.random.randn(out_channels, in_channels, kernel_size,
kernel_size) * std
self.bias = np.zeros(out_channels, dtype=np.float32)
```

Why: Kaiming/He initialisation reduces vanishing/exploding gradient tendencies in ReLU networks by scaling weights relative to the fan-in.

Forward Pass

File: `layers.py` — Lines 46–81

Equation — discrete convolution:

$$y[b,oc,i,j] = \Sigma_\{ic,u,v\}\ x[b, ic, i{\cdot}s+u, j{\cdot}s+v] \cdot w[oc, ic, u, v] + b[oc]$$

Key dimensions:

- out_H = (H − k) // s + 1
- out_W = (W − k) // s + 1

Why convolution: Learns local spatial features (edges, shapes) that are translationally invariant — particularly effective for digit images.

Backward Pass and Weight Update

File: `layers.py` — Lines 83–144

The backward pass uses the chain rule to compute three quantities:

- d_weights — gradient w.r.t. kernel weights
- d_bias — gradient w.r.t. bias
- d_inputs — gradient w.r.t. inputs (passed back to earlier layers)

Gradient descent update rule:

$$w \leftarrow w - \eta \cdot (\partial L/\partial w) \qquad b \leftarrow b - \eta \cdot (\partial L/\partial b)$$

Update code:

```
self.weights -= learning_rate * d_weights
self.bias -= learning_rate * d_bias
```

2.2 Flatten

File: `layers.py` — Lines 147–158

Forward: Reshapes `(N, C, H, W)` → `(N, C·H·W)` so the feature maps can feed into the fully-connected LinearLayer.

Backward: Simply reshapes the upstream gradient back to the original spatial shape.

```
def forward(self, inputs):
    self.inputs_shape = inputs.shape
    return inputs.reshape(inputs.shape[0], -1)
def backward(self, d_outputs):
    return {"d_out": d_outputs.reshape(self.inputs_shape)}
```

2.3 LinearLayer (Fully Connected)

Initialization

File: `layers.py` — Lines 160–173

Weight initialisation formula:

$$std = \sqrt{(2 / fan_in)} \quad where\ fan_in = in_features$$

```
self.weights = np.random.randn(out_features, in_features) * std
self.bias = np.zeros(out_features, dtype=np.float32)
```

Forward Pass

File: `layers.py` — Lines 174–188

Equation:

$$y = x \cdot W^T + b$$

Weights are shaped `(out_features, in_features)`, so `weights.T` becomes `(in_features, out_features)`.

```
return inputs @ self.weights.T + self.bias
```

Backward Pass and Weight Update

File: `layers.py` — Lines 190–226

Gradients:

```
d_weights = d_outputs.T @ inputs
d_bias = np.sum(d_outputs, axis=0)
d_inputs = d_outputs @ self.weights
```

Weight update (gradient descent):

```
self.weights -= learning_rate * d_weights
self.bias -= learning_rate * d_bias
```

3. Loss Functions (modules/loss.py)

3.1 CrossEntropy

Forward Pass

File: `loss.py` — Lines 17–41

Equation — for one-hot targets t and predicted probabilities p:

$$L = -(1/N) \cdot \Sigma_n \, \Sigma_c \;\; t_\{n,c\} \cdot log(p_\{n,c\})$$

Code:

```
inputs_clipped = np.clip(inputs, self.eps, 1.0 - self.eps)
loss = -np.sum(targets * np.log(inputs_clipped)) / inputs.shape[0]
```

Why clip with eps: Prevents log(0) and stabilises gradients when predicted probability approaches zero.

Backward Pass

File: `loss.py` — Lines 43–60

Gradient w.r.t. predicted probabilities p:

$$\partial L / \partial p_\{n,c\} = -(1/N) \cdot t_\{n,c\} \,/\, p_\{n,c\}$$

Code:

```
inputs_clipped = np.clip(self.inputs, self.eps, 1.0 - self.eps)
d_out = (self.targets / inputs_clipped) / self.inputs.shape[0] * -1
return {"d_out": d_out}
```

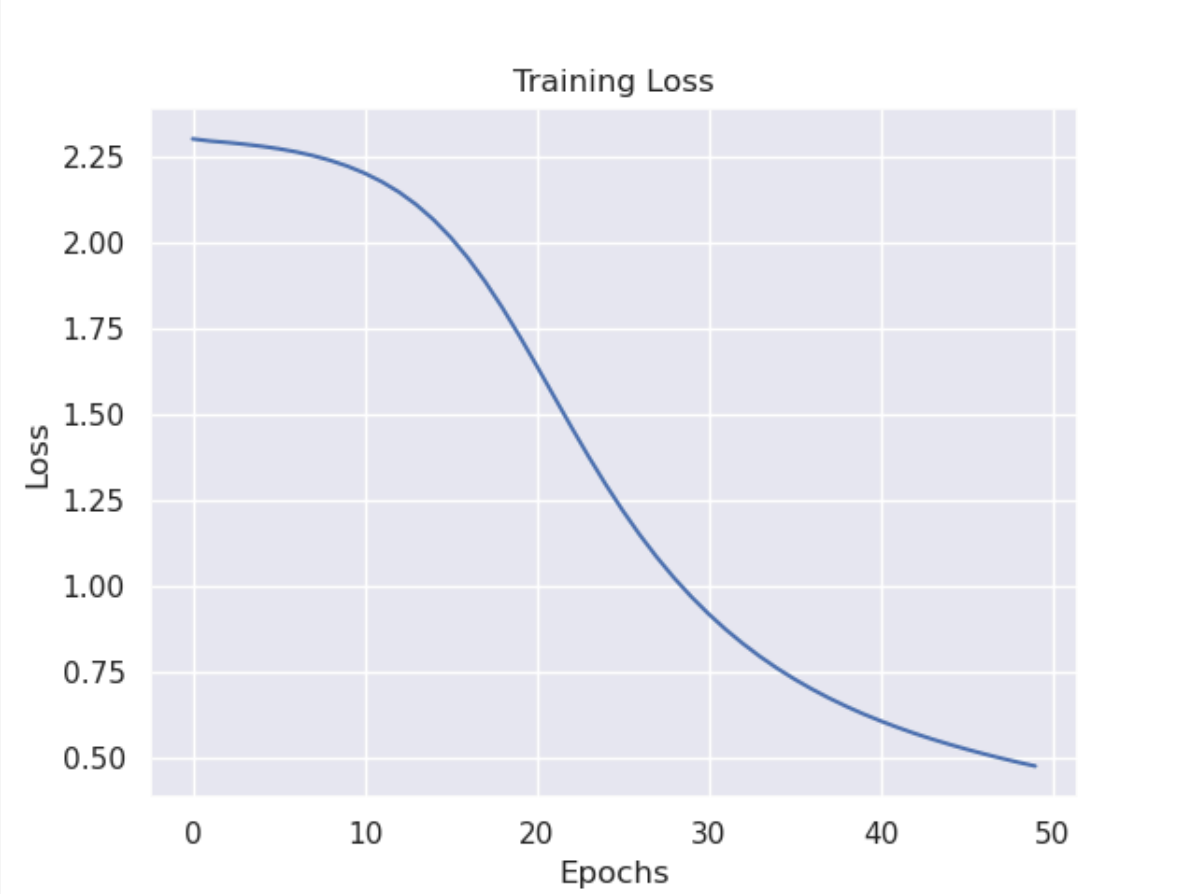
This gradient then flows back through the Softmax layer, which transforms it into gradients w.r.t. the logits.

4. Training Results (figures/)

4.1 Training Loss Curve

File: `figures/training_loss.png`

The training loss decreases smoothly from approximately 2.30 at epoch 1 down to approximately 0.48 by epoch 50, indicating stable training with no divergence or instability.

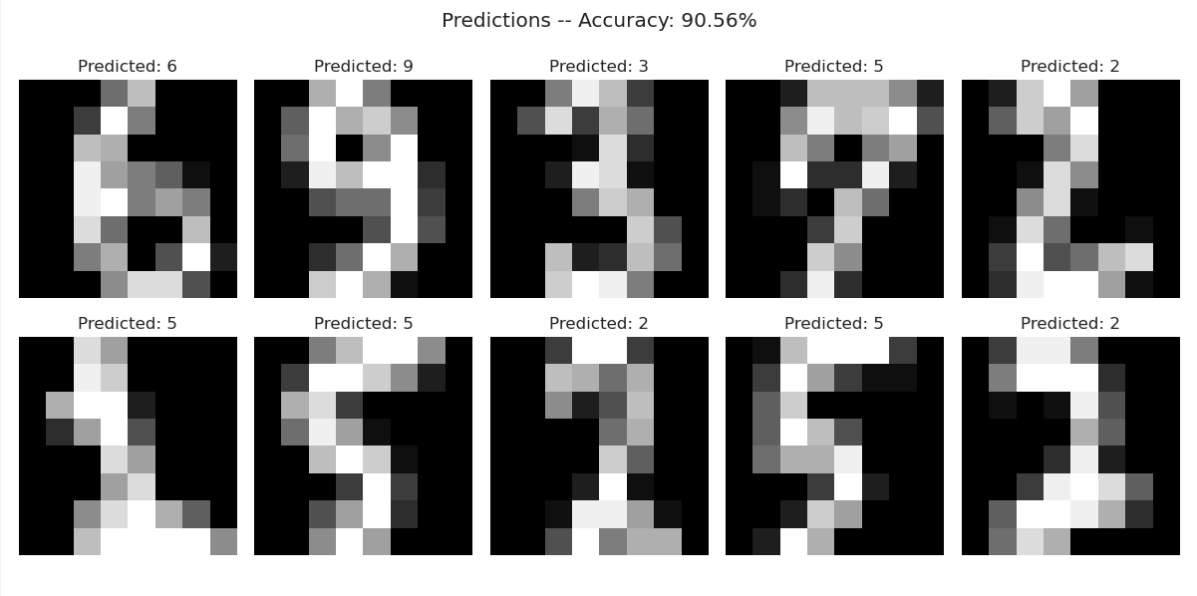


[Figure: training_loss.png]

4.2 Predictions

File: `figures/predictions.png`

The figure shows sample digit predictions with the overall model accuracy of 90.56%. The shown digit samples are largely correct and consistent with the high overall accuracy.



[Figure: predictions.png]

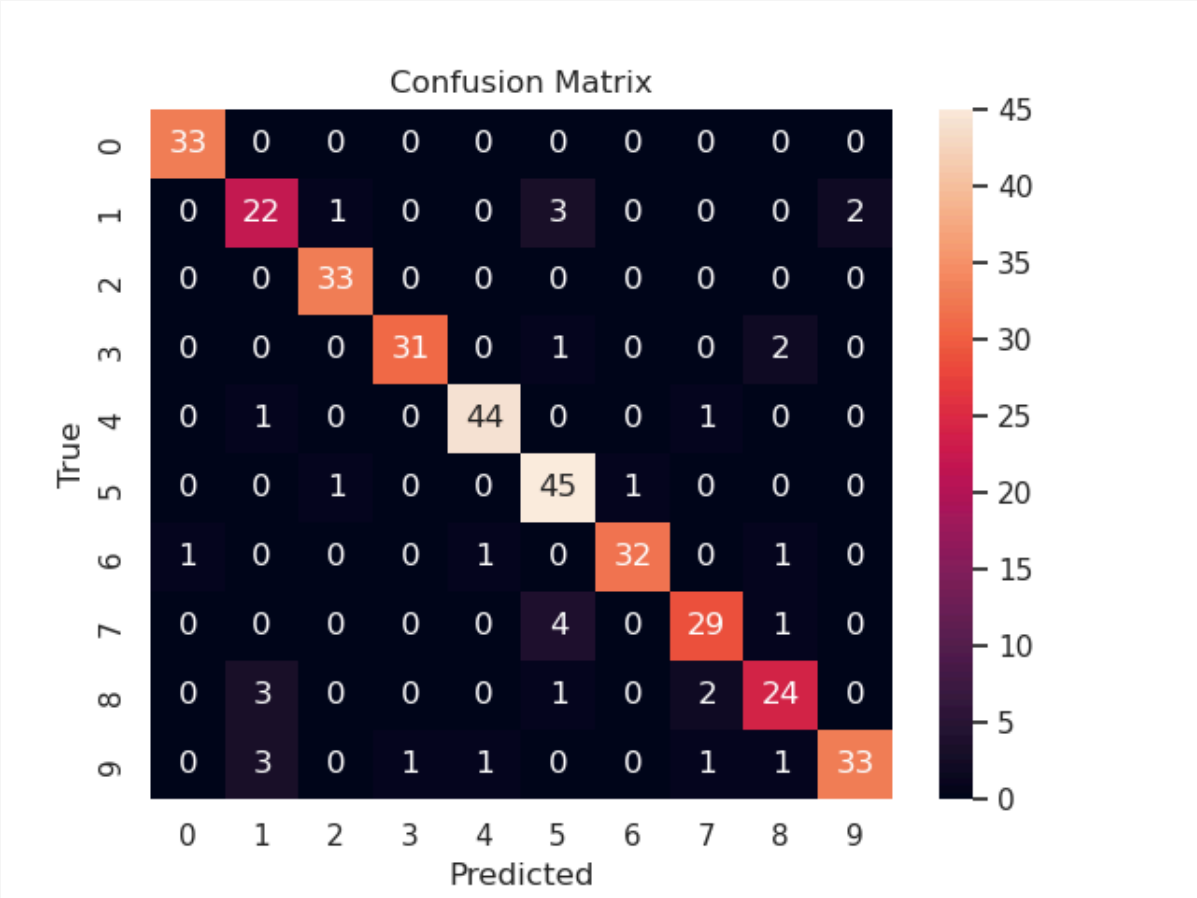
4.3 Confusion Matrix

File: `figures/confusion_matrix.png`

The matrix shows strong diagonal dominance — the vast majority of digits are correctly classified. Off-diagonal (misclassified) entries are small relative to the diagonal.
Per-class correct counts from the confusion matrix:

Digit	Correct Count	Notes
0	33	—
1	22	Low
2	33	—
3	31	—
4	44	High
5	45	High
6	32	—
7	29	—
8	24	Low

9	33	—



[Figure: confusion_matrix.png]

5. Hyperparameter Analysis & Best Results

5.1 Training Configuration

Best run: Epochs = 50, Learning Rate = 0.1, Batch Size = 32

Terminal output (key metrics):

```
Training config: Epochs=50, Learning Rate=0.1, Batch Size=32
Epoch:  1, Loss: 2.302414
Epoch: 50, Loss: 0.475613
```

```
Accuracy: 90.56%
```

5.2 Experiment Comparison

Epochs	Learning Rate	Accuracy	Notes
50	**0.1**	**90.56%**	Best result
25	0.1	32.50%	Too few epochs
25	0.01	15.56%	LR too small
25	0.01	24.44%	Second run, same config
25	1.0	64.17%	Higher LR helps but still underfits

5.3 Conclusion & Recommendations

Best observed settings: EPOCHS = 50 and LEARNING_RATE = 0.1 → 90.56% accuracy.

Reason: The loss curve is still decreasing at epoch 50, and experiments clearly show that reducing epochs to 25 dramatically harms accuracy even when LR remains 0.1.

Next steps to exceed 90.56% (without changing model architecture):

- Try epochs in the range 60–80 with LR = 0.1 and monitor whether accuracy improves or plateaus.
- Consider learning rate scheduling (e.g. decay after epoch 30) to fine-tune convergence.
- Add data augmentation to improve generalisation.